SYSTEM DESIGN

Rate Limiting Visual Guide

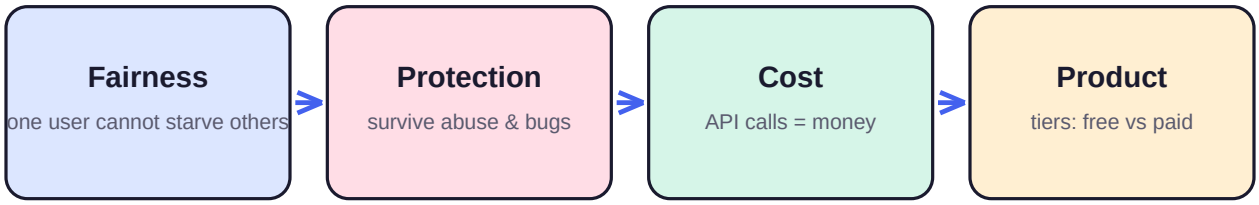
Protect shared resources:
buckets, windows, distributed counters, and headers.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Why Limit

Every shared resource has a capacity; limits turn overload into a polite 'no'.

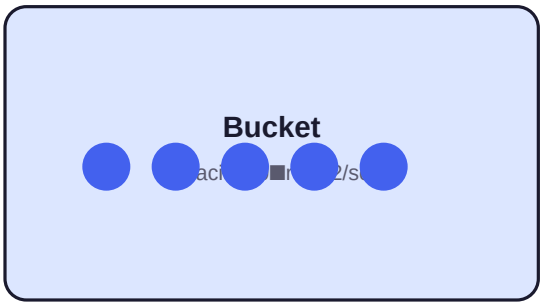


Limit at the EDGE (gateway/CDN) so rejected traffic never reaches expensive backends.

STEP 2/6

Token Bucket

Tokens refill over time; each request spends one. Bursts allowed up to bucket size.



request arrives:

- token available -> allow, remove token
- bucket empty -> reject (429)

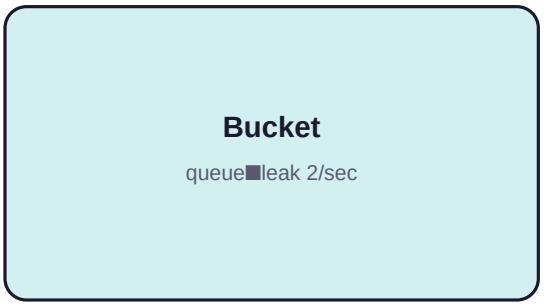
Steady rate 2/sec, but a burst of 10 passes instantly.

- Two knobs: RATE (sustained) and BURST (bucket size) -- tune them separately.
- The classic for APIs: friendly to bursty human traffic, strict on sustained abuse.

STEP 3/6

Leaky Bucket

Requests queue and drain at a FIXED rate -- smooth output, no bursts downstream.



burst of 20 arrives:

- queued, drained at exactly 2/sec
- queue full -> reject the excess

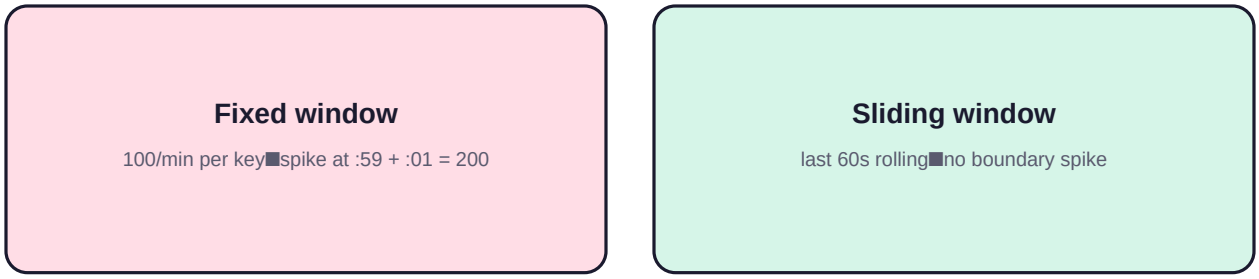
Use when the CONSUMER needs smooth flow (webhooks, worker

Token bucket shapes the SENDER (allow bursts); leaky bucket shapes the RECEIVER (enforce smooth).

STEP 4/6

Fixed vs Sliding Window

Counting requests per time window -- watch the boundary.

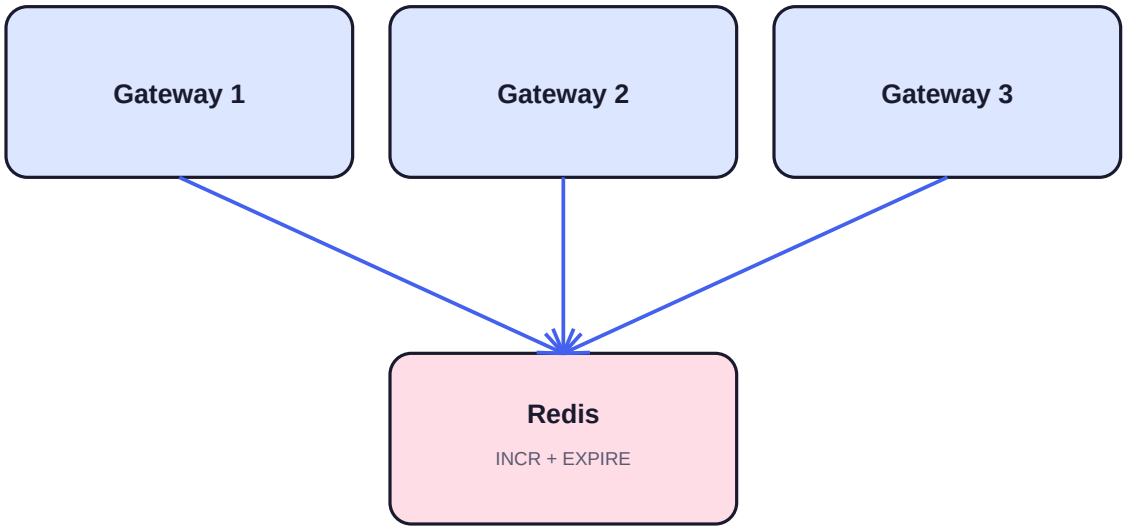


- Fixed window flaw: 100 requests at 12:00:59 + 100 at 12:01:01 = 200 in 2 seconds, all 'allowed'.
- Sliding window log: exact, but stores every timestamp. Sliding window counter: approximate, cheap (Redis-friendly).
- Fixed windows are fine for coarse quotas (per-day API plans); use sliding for per-second protection.

STEP 5/6

Distributed Limiting with Redis

One counter per key, shared by all gateway replicas.




```
count = INCR rl:{user}:{minute}  |  EXPIRE ... 60  |  allow if count <= 100
```

Lua script the check+increment atomically. For huge scale, shard counters or accept approximate local limits.

STEP 6/6

Communicate the Limit

A limit clients can see is a limit they respect.

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 37
X-RateLimit-Reset: 1718006400

429 Too Many Requests
Retry-After: 42
```

- Send quota headers on EVERY response, not just rejections -- clients budget proactively.
- 429 + Retry-After is the contract; exponential backoff on the client completes it.
- Log rejections per key: your best abuse-detection signal lives in the 429 stream.

STEP ✓/✓

Key Takeaways

Rate Limiting — Remember

- Limit at the edge for fairness, protection, cost control, and tiered products.
- Token bucket: refill over time, allows bursts up to capacity -- the API classic.
- Leaky bucket: fixed drain rate, smooths bursts -- for consumers that need steady flow.
- Fixed windows spike at boundaries; sliding windows cost more but stay accurate.
- Share counters in Redis (atomic Lua), and always tell clients the quota via headers + 429.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev